Data Engineering Labs Reflection Report (Lab 1 & Lab 2)

Groupe members: ***Mohamed LAABYDY*, ***Ali EL BAHA***

*** Ecole Centrale Casablanca**

Supervisors : Lamia BEN HIBA

Année académique: 2025-2026

February 21, 2026

Abstract

This report documents the design, implementation, and critical evaluation of a two-phase data engineering project centred on the Google Play Store ecosystem. Lab 1 established a Python-driven ingestion pipeline that extracts application metadata and user reviews, landing raw artefacts in JSON format. Lab 2 elevated this foundation into a fully modelled analytics platform by introducing dbt transformations over DuckDB, encompassing staging, dimensional, fact, and serving layers alongside native data-quality tests, incremental materialisation, and SCD Type 2 versioning via snapshots. The report reflects on architectural decisions, fragility points, and the most impactful lessons learned across both phases.

Contents

1. Final Architecture (Implemented)

Our final pipeline is split into two clearly separated layers: (1) Python ingestion + raw landing, and (2) dbt + DuckDB transformations, tests, and serving marts.

1.1. Raw ingestion and landing (Python / Lab 1)

src/scrapper.py extracts Google Play metadata and reviews and writes raw JSON.
Raw file used in Lab 2 staging: dbt/data/raw/ai_note_apps_with_reviews.json.

1.2. Transformation and modeling (dbt / Lab 2)

Staging:

- `dbt/models/staging/stg_playstore_apps.sql`
- `dbt/models/staging/stg_playstore_reviews.sql`

Dimensions: `dim_developers.sql`, `dim_categories.sql`, `dim_apps.sql`, `dim_date.sql`, `dim_apps_scd.sql`.
Fact: `dbt/models/marts/facts/fact_reviews.sql` (incremental).
Serving: `srv_monthly_rating_trend.sql` and `srv_developer_performance.sql` for BI consumption.
Snapshot SCD2: `dbt/snapshots/apps_scd_snapshot.sql`.

2. Implementation Details

2.1. Incremental Loading (E.1)

We converted `fact_reviews` from table to incremental with `unique_key = review_id` in `dbt/models/marts/facts/fact_reviews.sql`. The incremental filter loads only rows where `review_timestamp` is greater than `max(review_timestamp)` already present in the target table.

- Materialization: incremental
- Unique key: `review_id`
- Incremental predicate: `review_timestamp > max(existing review_timestamp)`

Observed behavior during test: first rerun after appending a review added exactly one row; next rerun was idempotent (no duplicate insertion).

2.2. SCD Type 2 (E.2)

We implemented SCD2 on apps using dbt snapshots with check strategy on business attributes (`title`, `developer_name`, `category_name`, `app_score`, `ratings`, `installs`, `price`). This creates versioned rows with `dbt_valid_from`/`dbt_valid_to`.

- Snapshot model: `dbt/snapshots/apps_scd_snapshot.sql`
- Historical dimension model: `dbt/models/marts/dimensions/dim_apps_scd.sql`
- Current row flag: `is_current = (dbt_valid_to is null)`
- Fact historical join: `fact_reviews` now joins to `dim_apps_scd` with validity-window conditions.

In our controlled test, changing one app category in raw source and rerunning snapshot produced two versions for the same `app_id` (one closed + one current), confirming SCD2 behavior.

2.3. Data Quality and Testing

We used dbt schema tests at each layer for structural integrity, referential integrity, value-domain control, and uniqueness.

- **Staging tests (schema.yml):** keys not_null/unique, relationships app_id, accepted values for ratings.
- **Dimensions tests:** PK uniqueness, FK relationships (`dim_apps` -> `dim_developers`/`dim_categories`), date uniqueness.
- **Fact tests:** review uniqueness, FK relationships to dimensions, accepted rating values.

- **Serving tests:** not_null and uniqueness on business output keys (e.g., `year_month`, `developer_key`).

Main test runs completed successfully on our models (staging, dimensions, facts, serving, and SCD models).

3. Python-Only vs dbt-Based Pipeline Comparison

Dimension	Lab 1 (Python-only)	Lab 2 (dbt + DuckDB)	Observed Impact
Transformation logic	Embedded in Python scripts	SQL models by layer (staging/marts/serving)	Higher readability and maintainability in Lab 2
Testing	Manual checks + custom script	Native dbt tests + relationship checks	Faster detection of schema/key issues
Lineage	Implicit in code flow	Explicit via `ref()` graph	Safer refactoring and easier onboarding
History management	Not native	Native snapshots for SCD2	Version tracking became systematic
Performance strategy	Mostly full refresh	Incremental materialization supported	Reduced recomputation and better scalability potential

Table 1: Python-only vs dbt-based pipeline comparison

4. Reflection

4.1. Most Fragile Part of the Pipeline

The most fragile part was historical temporal joining after SCD2 introduction. Because snapshot validity timestamps are runtime-driven, event timestamps in reviews (historical dates) can fall outside validity windows if the snapshot baseline is not aligned to source change timestamps. This makes historical attribution sensitive and can silently produce empty/underfilled facts.

4.2. Biggest Architectural Insight

The key insight is that separating ingestion, transformation, and serving is not only about clarity; it directly improves reliability. In dbt, each model has one grain and one responsibility. This made debugging and controlled evolution (incremental + SCD2) much easier than in one monolithic Python flow.

4.3. One Design Decision We Would Change

We would redesign SCD2 validity anchoring using a trustworthy source-side `updated_at` (or change timestamp). Check-strategy snapshots without source time semantics are fine for change tracking, but weak for strict event-time historical joins. A source timestamp strategy would make fact-to-dimension temporal joins more robust.

5. Specific References to Our Models and Outputs

- `dbt/models/staging/stg_playstore_apps.sql`
- `dbt/models/staging/stg_playstore_reviews.sql`
- `dbt/models/marts/dimensions/dim_developers.sql`
- `dbt/models/marts/dimensions/dim_categories.sql`
- `dbt/models/marts/dimensions/dim_apps.sql`
- `dbt/models/marts/dimensions/dim_date.sql`
- `dbt/models/marts/dimensions/dim_apps_scd.sql`
- `dbt/models/marts/facts/fact_reviews.sql`
- `dbt/models/marts/serving/srv_monthly_rating_trend.sql`
- `dbt/models/marts/serving/srv_developer_performance.sql`
- `dbt/snapshots/apps_scd_snapshot.sql`

Current table counts (latest run): `stg_playstore_apps`=94, `stg_playstore_reviews`=6416, `dim_developers`=90, `dim_categories`=5, `dim_apps`=94, `dim_date`=4092, `dim_apps_scd`=95, `fact_reviews`=0, `srv_monthly_rating_trend`=105, `srv_developer_performance`=84.
SCD split currently visible: current rows=94, historical rows=1 in `dim_apps_scd`.
Date coverage: 2014-11-23 to 2026-02-04 (`dim_date`).

6. Screenshots and Interpretation

6.1. Monthly Average Rating Trend

This chart validates that time-based serving is consumable. We can track rating evolution month by month and detect periods of degradation or recovery.

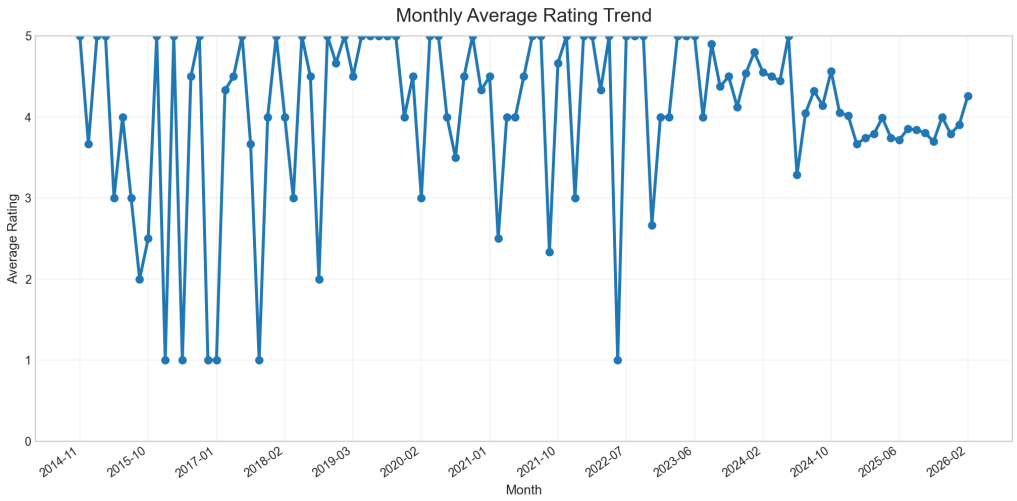

Figure 1: Monthly Average Rating Trend

6.2. App Performance: Popularity vs Satisfaction

This scatter combines volume and quality in one view: x-axis is review volume, y-axis is average rating, point size scales with volume, and color reflects low-rating pressure. It helps detect "high-volume but low-satisfaction" outliers.

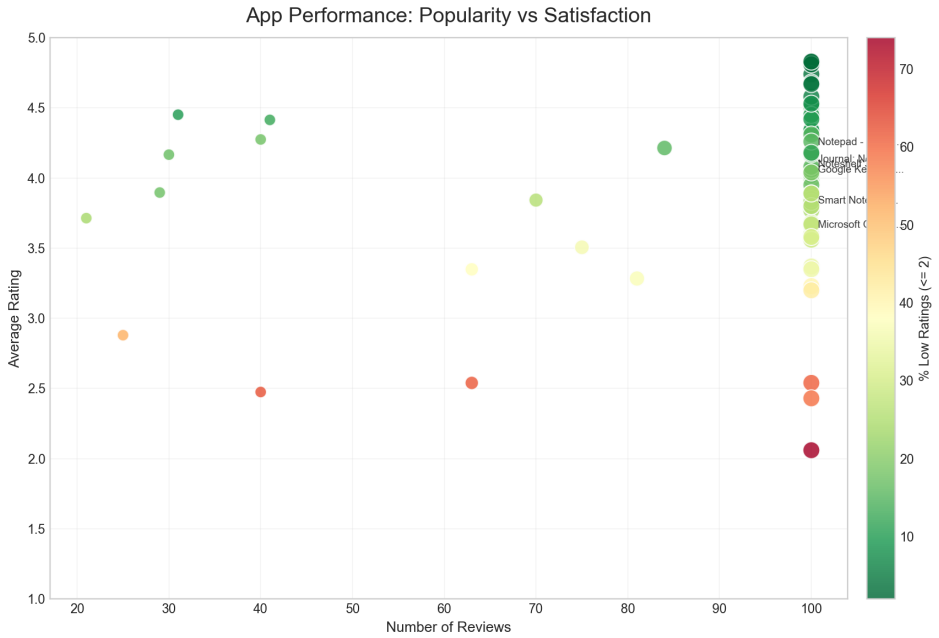

Figure 2: App Performance: Popularity vs Satisfaction

6.3. Apps with Highest Percentage of Low Ratings

This bar chart prioritizes risk from a product perspective. Ranking by low-rating percentage is more actionable for triage than average rating alone, especially when review volume is controlled by a minimum threshold.

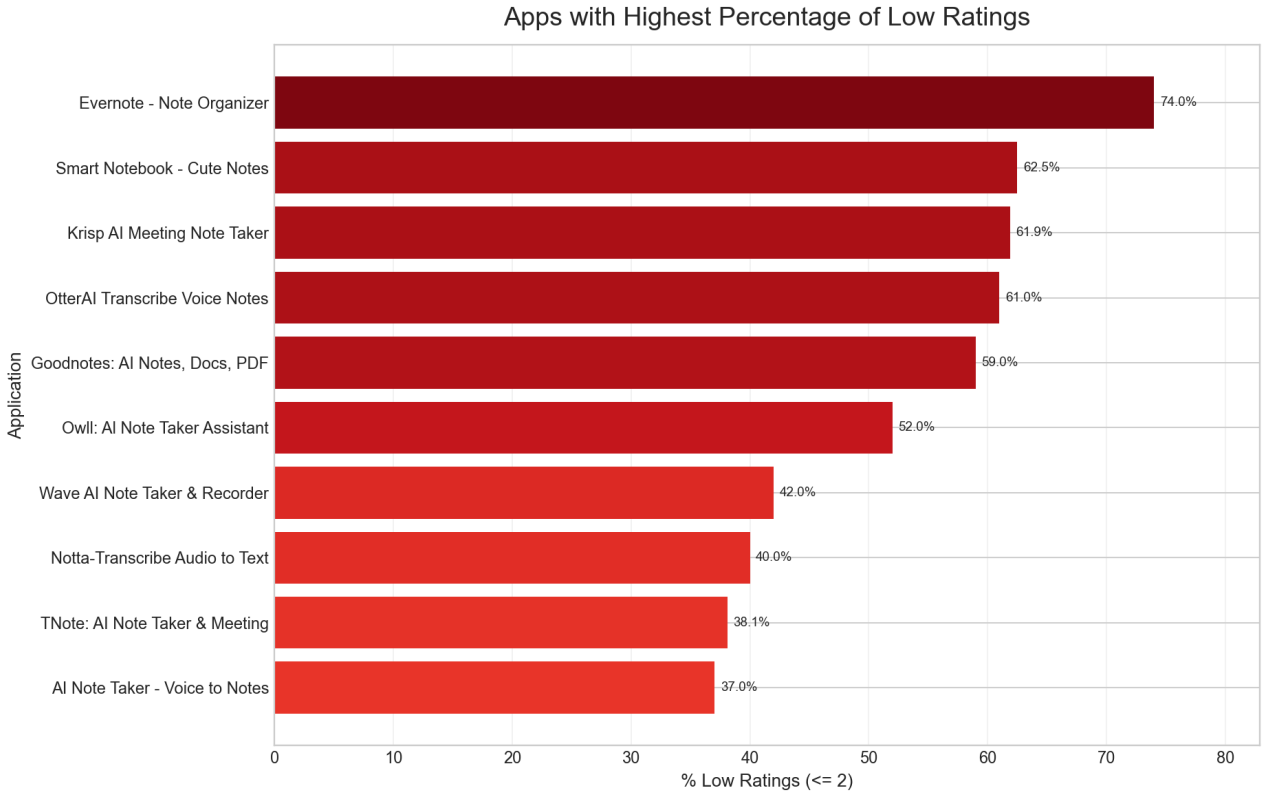

Figure 3: Apps with Highest Percentage of Low Ratings

7. Conclusion

Lab 1 gave us end-to-end ownership of ingestion and transformation logic. Lab 2 transformed this into a structured analytics platform with tested layers, reproducibility, incremental loading, and SCD2 versioning. The most valuable outcome was not only code quality, but architectural clarity and controlled change handling.